



**BERLIN SCHOOL OF
BUSINESS & INNOVATION**

**Essay / Assignment Title: Deep Learning Driven Image
Segmentation for Enhanced Diagnostic Accuracy in Medical
Imaging**

Programme Title: MSc Data Analytics

Name: Aadhithya Roy Payaroli

Year: 2026

CONTENTS

<u>ABSTRACT</u>	<u>4</u>
<u>1. INTRODUCTION</u>	<u>5</u>
<u>2. LITERATURE REVIEW</u>	<u>6</u>
<u>3. PROBLEM FORMULATION</u>	<u>7</u>
<u>4. METHODOLOGY</u>	<u>8</u>
<u>5. MODEL ARCHITECTURES</u>	<u>11</u>
<u>6. IMPLEMENTATION (PYTHON + TENSORFLOW)</u>	<u>12</u>
<u>7. MODEL EVALUATION</u>	<u>15</u>
<u>8. CONCLUSION</u>	<u>21</u>
<u>9. REFERENCES</u>	<u>22</u>
<u>10. LIST OF TABLES AND FIGURES</u>	<u>23</u>
<u>APPENDIX</u>	<u>24</u>

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the program).

Name and Surname (Capital letters): AADHITHYA ROY PAYAROLI

Date: 30 – January – 2026

ABSTRACT

Correct segmentation of tumor regions in medical images helps with diagnosis, treatment planning, and checking changes over time. This project builds an automatic pipeline to segment brain tumors in MRI scans. It uses the BRISC 2025 dataset (Kaggle, 2025), which includes expert-annotated masks.

We take the standard U-Net model as the baseline. Then we compare it with two modified U-Net versions. In these versions, we replace the encoder with pretrained backbones: MobileNetV2 and ResNet50. The aim is to check the trade-off between segmentation performance and model scalability (number of parameters and inference time).

On the held-out test set, the U-Net with ResNet50 backbone gives the best scores: Dice = 0.8820 and IoU = 0.7889. The U-Net with MobileNetV2 backbone reaches Dice = 0.8692. This version has much fewer parameters and the lowest inference time of 5.72 ms per image.

The results show that replacing the encoder with a pretrained backbone is a useful way to improve tumor segmentation when there are limits on model size or speed.

1. INTRODUCTION

Medical images like MRI, CT, and ultrasound provide spatial data. Doctors use this data to study, find and cure diseases (Adoui et al., 2019). However, identifying areas like tumor borders is not very easy. It requires manual segmentation. This process is a time consuming activity. It is also hard to standardize. Different experts mark the same region in various ways (Adoui et al., 2019). The problem worsens when borders are unclear, when tissue looks mixed, or when there is image noise.

Automatic segmentation addresses these issues. It creates pixel-level masks from images. This method allows for measuring volume, shape, and growth rate. It also improves consistency across patients, imaging centers and various types of tumor (Adoui et al., 2019; Abdelrahman and Viriri, 2023). Deep learning is now the main method for medical image segmentation. Encoder-decoder networks with convolutions learn context and detail from image and mask pairs (Hussain and Shouno, 2024). U-Net is one of the standard models that's used for this job. Like the name, it has a U structure. It uses skip connections to keep spatial details from early layers. The encoder learns high-level features (Adoui et al., 2019). Even with strong performance, segmentation models must meet practical needs. These include low parameter counts, fast inference time, and compatibility with clinics or devices that have limited resources (Ronneberger et al., 2015). Research shows that design choices, like skip connections, influence model performance when image texture varies across scanners and protocols (Jiang et al., 2019; Hussain and Shouno, 2024).

Ethical issues are also a problem. Datasets could be biased toward certain patient groups or some scanners. So, we must make sure that the models work well for all patients or types of scanners etc (Li et al., 2025). The goal of this work is to implement and compare segmentation models for marking tumors in medical images. We aim for a trustworthy identification of tumor. We use accuracy of the models using Dice, IoU, Precision, and Recall. We will also evaluate scalability through parameter count and inference speed (Abdelrahman and Viriri, 2023; Ronneberger et al., 2015).

2. LITERATURE REVIEW

U-Net is the most common baseline model for medical image segmentation (Ronneberger et al., 2015). It has an encoder-decoder structure with skip connections. These skip connections keep spatial details from the input image. The encoder captures context at different levels. This structure makes U-Net a standard choice for tasks like tumor segmentation (Adoui et al., 2019).

Some extensions of U-Net add attention mechanisms. These mechanisms help the model focus on important structures (Hussain and Shouno, 2024). In recent works, people use pretrained encoders in place of the original encoder. These encoders include networks like ResNet50 or EfficientNet (Abdelrahman and Viriri, 2023). When ResNet50 is used as the backbone in U-Net for brain tumor segmentation on MRI images, it can reach Dice scores of 0.87 or higher. This gives a balance between accuracy and computation (Adoui et al., 2019). Deeper backbones like ResNet50 improve feature extraction, but they increase the number of parameters and inference time (Hussain and Shouno, 2024).

Many studies compare different backbones in U-Net, such as MobileNetV2 and ResNet50 (Abdelrahman and Viriri, 2023). MobileNetV2 has fewer parameters and faster inference time. This makes it good for devices with limited resources. ResNet50 usually gives higher Dice and IoU scores, but it needs more resources. This shows a trade-off between accuracy and practical use. Deeper encoders give better results, while lightweight encoders help in real applications (Ronneberger et al., 2015).

Newer models use transformers or hybrid designs. These capture long-range dependencies better than standard CNNs. This improves boundary accuracy on different datasets (Ronneberger et al., 2015). In real-world use, robustness matters a lot. Choices like skip connections affect performance when images come from different scanners or protocols (Jiang et al., 2019; Hussain and Shouno, 2024).

Most studies focus on high accuracy metrics like Dice and IoU. Few studies compare accuracy with scalability factors. Like parameter count and inference latency in the same setup. This work fills that gap. It evaluates U-Net with backbones - MobileNetV2 and ResNet50 on the same pipeline and same dataset (Abdelrahman and Viriri, 2023; Ronneberger et al., 2015).

3. PROBLEM FORMULATION

Let an input image be $x \in \mathbb{R}^{(H \times W \times C)}$ and the ground-truth binary tumor mask be $y \in \{0,1\}^{(H \times W)}$. The model learns a function $f_\theta(x) = \hat{y}$ producing a probability map $p \in [0,1]^{(H \times W)}$, then binarize at inference with threshold $t = 0.5$ (fixed across models) to obtain $\hat{y}$.

The main goal of optimization is to get high overlap between the ground-truth y and the predicted $\hat{y}$. We measure this overlap using Dice coefficient and IoU (Milletari et al., 2016).

We also want to control two types of errors that have clinical importance:

False negatives: tumor pixels marked as non-tumor. These are more risky because they can miss parts of the tumor (Adoui et al., 2019)..

False positives: non-tumor pixels marked as tumor. These cause over-segmentation and may add extra work in treatment planning or review (Adoui et al., 2019).

The model trains to maximize Dice and IoU while keeping these errors low and balanced for practical use in diagnosis and follow-up (Milletari et al., 2016).

4. METHODOLOGY

BRISC 2025 is a public dataset for brain tumor segmentation and classification using MRI scans. It includes expert-annotated masks (Kaggle, 2025). The dataset has 6,000 contrast-enhanced T1-weighted MRI images. These images come from different sources and belong to four classes: glioma, meningioma, pituitary tumor, and no tumor. Each image comes with a pixel-wise segmentation mask (Kaggle, 2025).

In this project, we only use the segmentation part of the dataset. The data is divided as follows:

- Training images: 3,146
- Validation images: 787
- Held-out test images: 860

This gives 3,933 images for training and validation. The remaining 860 images are kept separate for final testing (Milletari et al., 2016).

We create the training and validation sets by splitting the available training images in an 80/20 ratio. We use a fixed random seed (`random_state = 42`) and we do not use stratification. The test folder provided in the dataset is kept fully separate and used only for final evaluation. All images and their masks are resized to 256×256 pixels. This helps keep computation the same across models and allows fair comparison (Ronneberger et al., 2015). Images are read as grayscale (single channel) and converted to float32 values in the range $[0, 1]$. Masks are resized using nearest-neighbor interpolation to keep label boundaries correct. Then masks are scaled to $[0, 1]$ and converted to binary using a threshold of > 0.5 .

For models with MobileNetV2 and ResNet50 backbones, we convert grayscale images to RGB. By repeating the single channel three times. We then multiply by 255 to match ImageNet scaling. And then apply the specific `preprocess_input` function for the backbone (TensorFlow, 2025; Sandler et al., 2018; He et al., 2016).

We maintain correct image-mask pair. It's done by using fixed path mapping (image files end with `.jpg` and masks with `.png`). This ensures each image stays matched with its correct binary tumor mask during the entire process.

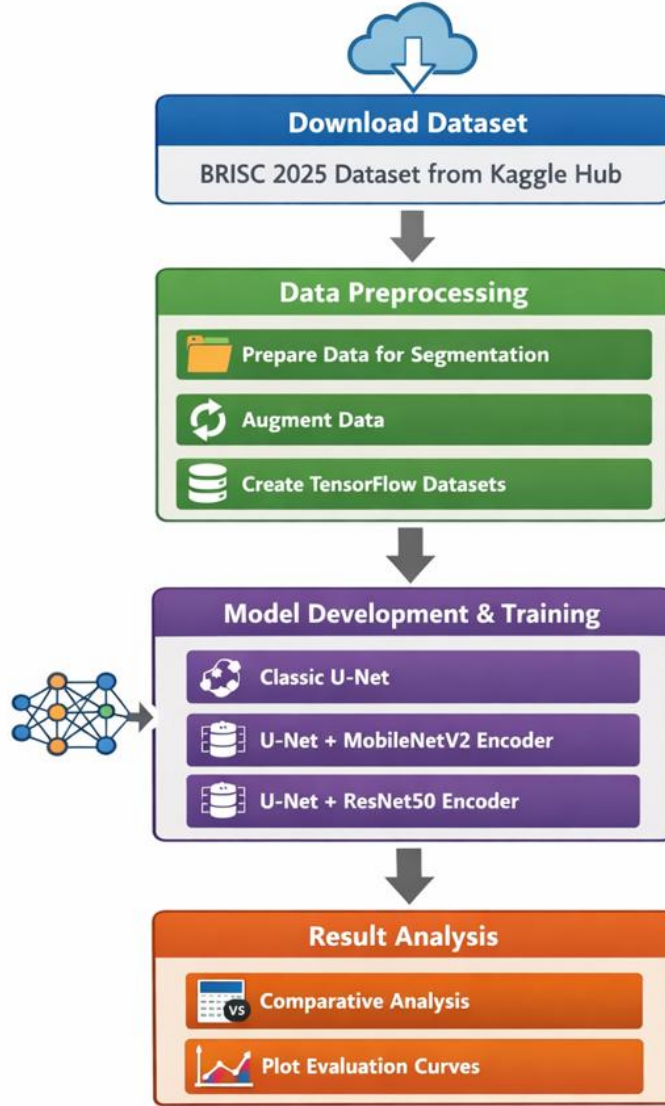


Figure 1: Proposed methodology of the project

During training, we apply the following augmentations to both images and masks:

- Random horizontal flip (probability = 0.5)
- Random vertical flip (probability = 0.5)
- Random 90° rotations (probability = 0.5; rotations of 0, 90, 180, or 270 degrees)
- Random brightness adjustment (max_delta = 0.2)
- Random contrast adjustment (range from 0.5 to 2.0)

These augmentations help the model handle differences in MRI scan acquisition. The changes remain realistic for MRI data (Milletari et al., 2016).

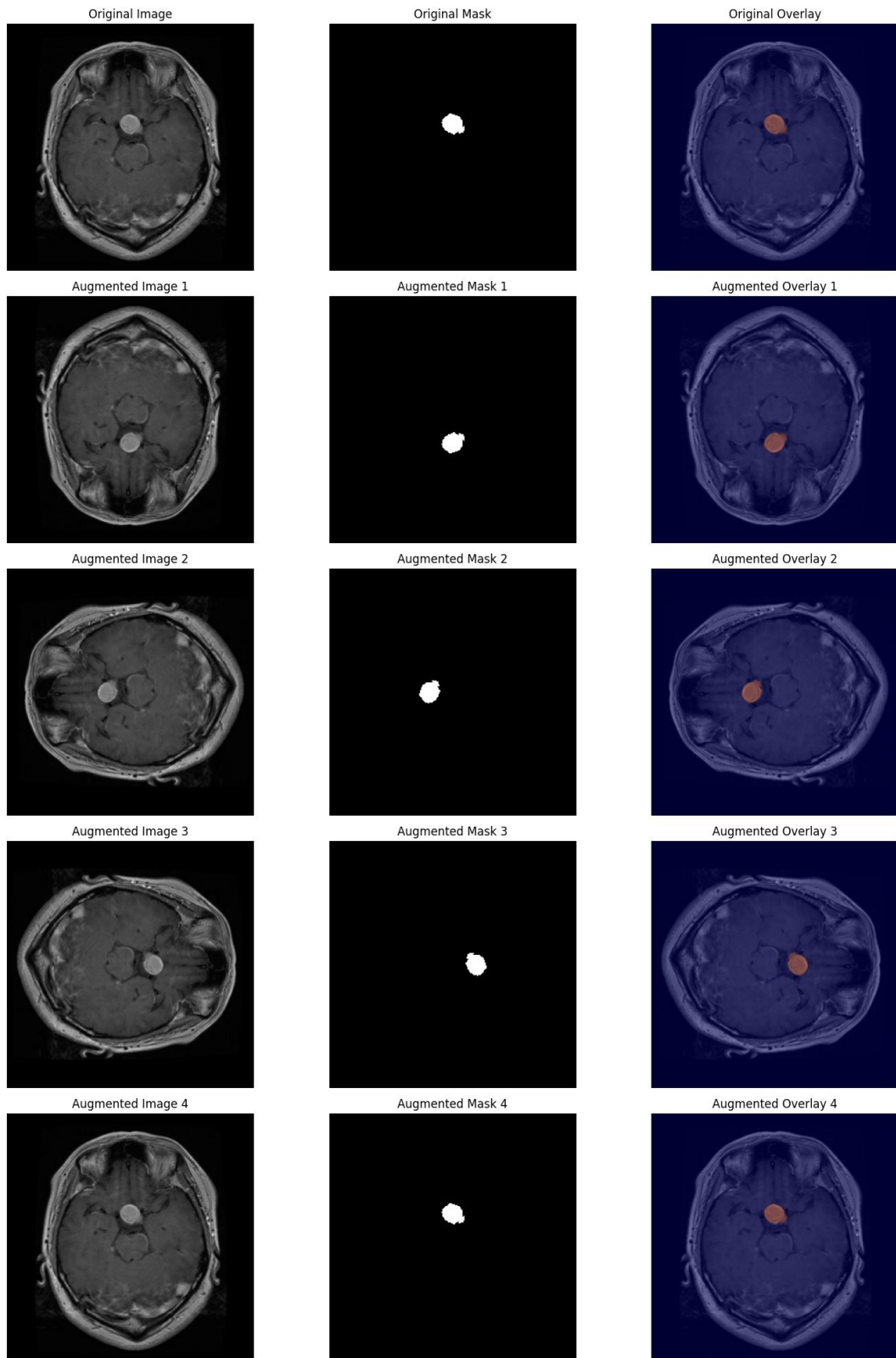


Figure 2: Augmentation Data Samples

5. MODEL ARCHITECTURES

All models use a 256×256 input image and bring out a single-channel 256×256 output. The output is a probability map from a sigmoid activation, where each pixel value (0 to 1) represents the chance it belongs to the tumor region (Ronneberger et al., 2015; Milletari et al., 2016).

Classic U-Net

This is the base model. It uses an encoder-decoder structure with skip connections. The encoder uses convolutional layers to reduce image size and increase channels. The decoder then upsamples these features, and, merges them with the encoder's skip connections. This helps preserve fine spatial details for tumor boundaries (Ronneberger et al., 2015). We use this as a baseline due to its popularity, common use and proven performance in medical segmentation related projects (Milletari et al., 2016).

U-Net with MobileNetV2 Encoder

In this version, we replace the original U-Net encoder with a pretrained MobileNetV2 encoder, keeping the decoder. The encoder weights are frozen. MobileNetV2 uses depthwise separable convolutions, which reduces parameters and computation (Sandler et al., 2018). The plan is to achieve good segmentation with fewer parameters and faster inference. Actually, this model is good for limited resource settings like mobile or edge devices (Abdelrahman and Viriri, 2023).

U-Net with ResNet50 Encoder

Here, we replace the encoder with a pretrained ResNet50, while we are keeping the U-Net decoder and freezing the encoder weights. ResNet50 uses residual blocks to enable stable training of deeper networks (He et al., 2016). It typically learns higher, also more detailed features compared to lighter encoders like the previous one. This often improves boundary detection and overlap scores (Dice and IoU) (Adoui et al., 2019). However, it requires more parameters and longer inference time. This version targets maximum segmentation accuracy when computational cost is not a major concern or issue (Ronneberger et al., 2015).

6. IMPLEMENTATION (PYTHON + TENSORFLOW)

The models are implemented using Python 3.12. TensorFlow 2.19 is used with the Keras API. Training and inference are performed on an NVIDIA A100-SXM4-80GB GPU with a batch size of 16.

For the MobileNetV2 and ResNet50 encoders, we load weights pretrained on ImageNet. We freeze them during training. Only the decoder part of the U-Net is trained. This approach reduces the number of trainable parameters. It speeds up training. This also lowers the risk of overfitting, and leverages useful features from the pretrained networks (Ronneberger et al., 2015; Sandler et al., 2018; He et al., 2016). Input grayscale MRI images are converted to three-channel RGB by repeating the single channel three times. The specific ImageNet preprocessing function (mean subtraction and scaling) for each backbone is applied. To ensure compatibility with the pretrained weights.

Training uses the Adam optimizer with a learning rate of $1e-4$. We monitor the validation Dice score. We implement EarlyStopping with a patience of 10 epochs. `restore_best_weights` is set to `True`. The best model is saved based on the validation Dice coefficient using a `ModelCheckpoint` callback. A `ReduceLROnPlateau` callback is also applied. Which reduces the learning rate by a factor of 0.5, if the validation Dice does not improve for 5 epochs. The maximum training epochs are set to 20. But early stopping usually ends training sooner (Prechelt, 2012). Since tumor regions are typically small in MRI images, creating a class imbalance, we use a combined loss function: Binary Cross-Entropy (BCE) + Dice loss. BCE provides stable pixel-wise gradients. Dice loss focuses on the overlap between prediction and ground truth. It handles the imbalance effectively. The final loss is the sum of BCE and Dice loss (Milletari et al., 2016). We calculate Dice coefficient as $(2 \times \text{intersection} + \text{smooth}) / (\text{true pixels} + \text{predicted pixels} + \text{smooth})$. Smooth value is $1e-6$ to avoid division by zero. Dice loss is $1 - \text{Dice coefficient}$ (Milletari et al., 2016).

```
def dice_coef(y_true, y_pred, smooth=1e-6):
    y_true_f = tf.keras.backend.flatten(y_true)
    y_pred_f = tf.keras.backend.flatten(y_pred)
    intersection = tf.keras.backend.sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (tf.keras.backend.sum(y_true_f) + tf.keras.backend.sum(y_pred_f) + smooth)

def dice_loss(y_true, y_pred):
    return 1 - dice_coef(y_true, y_pred)

def combined_loss(y_true, y_pred):
    bce = tf.keras.losses.binary_crossentropy(y_true, y_pred)
    return bce + dice_loss(y_true, y_pred)
```

Training times on the same hardware are:

- Classic U-Net: about 7 minutes
- U-Net + MobileNetV2: about 4 minutes
- U-Net + ResNet50: about 7 minutes

```
def benchmark_latency_ms_per_image(model, ds, warmup_batches=3, timed_batches=10):
    """
    Benchmarks inference latency in ms/image on a dataset.
    Uses warm-up to stabilize graph/CPU/GPU kernels.
    """
    # Grab a limited iterator (no shuffle)
    it = iter(ds)

    # Warm-up
    for _ in range(warmup_batches):
        imgs, _ = next(it)
        _ = model.predict(imgs, verbose=0)

    # Timed runs
    times = []
    n_images = 0

    for _ in range(timed_batches):
        imgs, _ = next(it)
        bs = imgs.shape[0]
        t0 = time.perf_counter()
        _ = model.predict(imgs, verbose=0)
        t1 = time.perf_counter()

        times.append(t1 - t0)
        n_images += int(bs)

    total_time = float(np.sum(times))
    ms_per_image = (total_time / n_images) * 1000.0

    # Variance across batches
    ms_per_batch = (np.array(times) * 1000.0)
    return {
        "Latency (ms/img)": float(ms_per_image),
        "Latency batch ms mean": float(ms_per_batch.mean()),
        "Latency batch ms std": float(ms_per_batch.std())
    }
```

To check scalability, we record two values for each model. First is the number of trainable parameters from the model summary. Second is inference latency in milliseconds per image. We run 3 warm-up batches to stabilize GPU and framework. Then we time batches using `model.predict()`. We calculate mean latency per image as total time divided by number of images. Standard deviation across batches to see variation is also calculated. This includes TensorFlow overhead and gives a practical comparison on the same hardware (Han et al., 2016; Ronneberger et al., 2015).

We set fixed random seeds for dataset splitting and shuffling to make results reproducible. For evaluation, we binarize predicted probability maps with threshold 0.5. We compute all metrics at pixel level on the full test set combined, not by averaging per-image scores.

7. MODEL EVALUATION

We evaluate all models on the held-out test set using standard segmentation metrics. The primary metrics are Dice coefficient and IoU, which measure overlap between predicted and true tumor masks. Dice is robust to class imbalance because it focuses on foreground overlap. IoU is stricter and often used alongside Dice (Milletari et al., 2016).

We also report supporting metrics:

- Precision: fraction of predicted tumor pixels that are correct (penalizes over-segmentation)
- Recall: fraction of true tumor pixels that are detected (penalizes missed tumor pixels)
- Pixel accuracy: overall correct pixels (reported but less informative because background pixels dominate)

We calculate all metrics after binarizing predictions at threshold 0.5 (Milletari et al., 2016)

7.1. Test results and scalability summary

Table 1: Results on the test set along with scalability measures

Model	Dice	IoU	Precision	Recall	Pixel Accuracy	Total Params	Trainable Params	Non-trainable Params	Latency (ms/img)
Classic U-Net	0.795	0.660	0.871	0.732	0.993	31030593	31030593	0	5.973
U-Net + MobileNetV2	0.869	0.769	0.908	0.834	0.995	15135745	12877761	2257984	5.718
U-Net + ResNet50	0.882	0.789	0.917	0.849	0.996	72368897	48781185	23587712	6.683

7.2. Advantages and disadvantages of each model

Classic U-Net

Advantages: Simple to implement; skip connections preserve fine spatial details; good baseline for biomedical tasks with limited data (Ronneberger et al., 2015).

Disadvantages: Higher parameter count (31 million); lowest Dice and IoU; weaker feature extraction without pretraining; less robust to texture variations across scanners (Jiang et al., 2019).

U-Net + MobileNetV2

Advantages: Low parameter count (15 million); fastest inference (5.72 ms per image); good balance between accuracy and efficiency; suitable for mobile or edge devices; pretrained features help faster convergence (Sandler *et al.*, 2018; Abdelrahman and Viriri, 2023).

Disadvantages: Slightly lower Dice and recall than deeper models; may not handle very complex tumors or strong texture variations as well due to lighter depth.

U-Net + ResNet50

Advantages: Highest Dice (0.882) and IoU (0.789); stronger features for variable tumor shapes and appearances; better handling of noise (He et al., 2016; Adoui et al., 2019).

Disadvantages: Highest parameter count (72 million) and latency (6.68 ms per image); higher risk of overfitting on smaller datasets; not ideal for low-resource settings (Han et al., 2016).

7.3. Discussion: quality vs. scalability

Both models with pretrained encoders (MobileNetV2 and ResNet50) clearly outperform the classic U-Net on Dice and IoU. This shows that using stronger pretrained backbones improves feature extraction and helps the model handle different tumor appearances better (Milletari et al., 2016).

For deployment:

- The ResNet50 version gives the best overlap scores but uses the most parameters and time.
- The MobileNetV2 version reaches near-top performance with much fewer parameters and the fastest speed. This makes it the most practical choice when memory or inference time is limited (Sandler *et al.*, 2018).

Robustness is important because MRI images from different scanners show texture and contrast changes. The skip-connection design in U-Net helps, but robustness should be tested on more diverse data beyond one split (Jiang et al., 2019).

7.4. Training and Validation Curves

Figures 3–4 show the training and validation Dice coefficient curves for the three models. The classic U-Net converges slowly and reaches a lower Dice score than the models that use pretrained encoders (Ronneberger et al., 2015; Tajbakhsh et al., 2018). Both the MobileNetV2-based and ResNet50-based models show fast improvement in the early epochs (Sandler et al., 2018; He et al., 2016). This shows the advantage of using pretrained feature representations (Tajbakhsh et al., 2018).

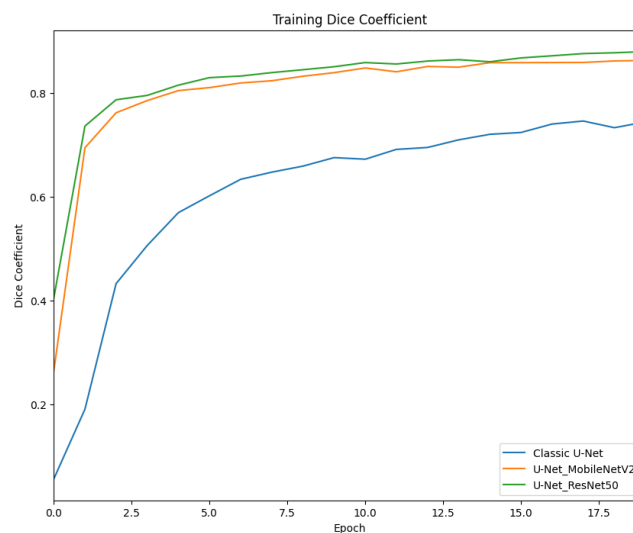


Figure 3: Training Dice Coefficient

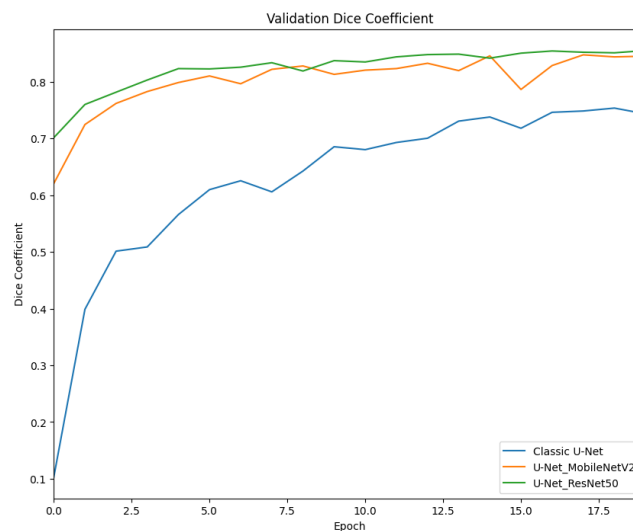


Figure 4: Validation Dice Coefficient

The ResNet50-based model gives the highest Dice scores on both training and validation sets during most of the training (He et al., 2016). This means it has stronger feature extraction ability (He et al., 2016). The MobileNetV2-based model performs close to ResNet50 (Sandler et al., 2018). It converges faster and shows less variation in the curves.

Figures 5–6 show the training and validation loss curves. All three models converge in a stable way and do not show strong signs of overfitting (Tajbakhsh et al., 2018). The models with pretrained encoders reach lower loss values much earlier than the classic U-Net (Tajbakhsh et al., 2018). The classic U-Net has higher loss values and slower decrease in loss (Ronneberger et al., 2015). For the MobileNetV2 model, small ups and downs appear in the validation loss at times (Sandler et al., 2018). However, the learning rate scheduler and early stopping keep these changes under control (Prechelt, 2018).

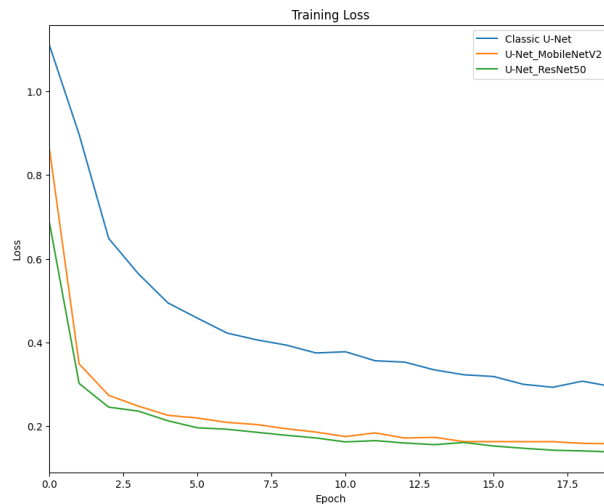


Figure 5: Training Loss

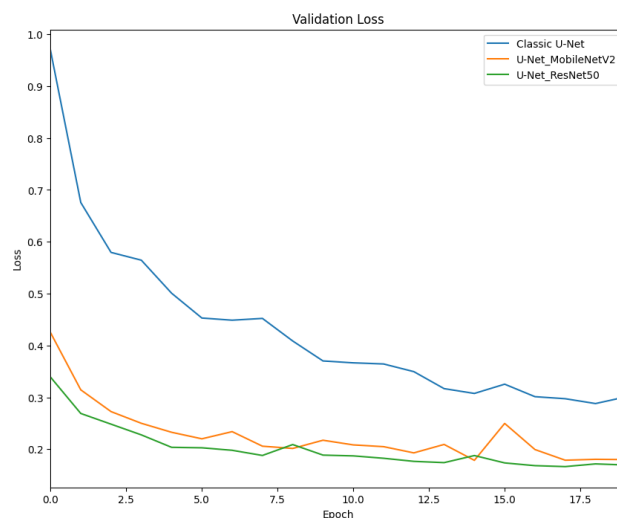


Figure 6: Validation Loss

7.5. Qualitative Segmentation Results

To add to the numbers, we show qualitative results on some randomly chosen test images (Tajbakhsh et al., 2018). For each example, we display four things: the original MRI slice, the ground-truth mask, the predicted binary mask, and the Dice score for that single image (Ronneberger et al., 2015).

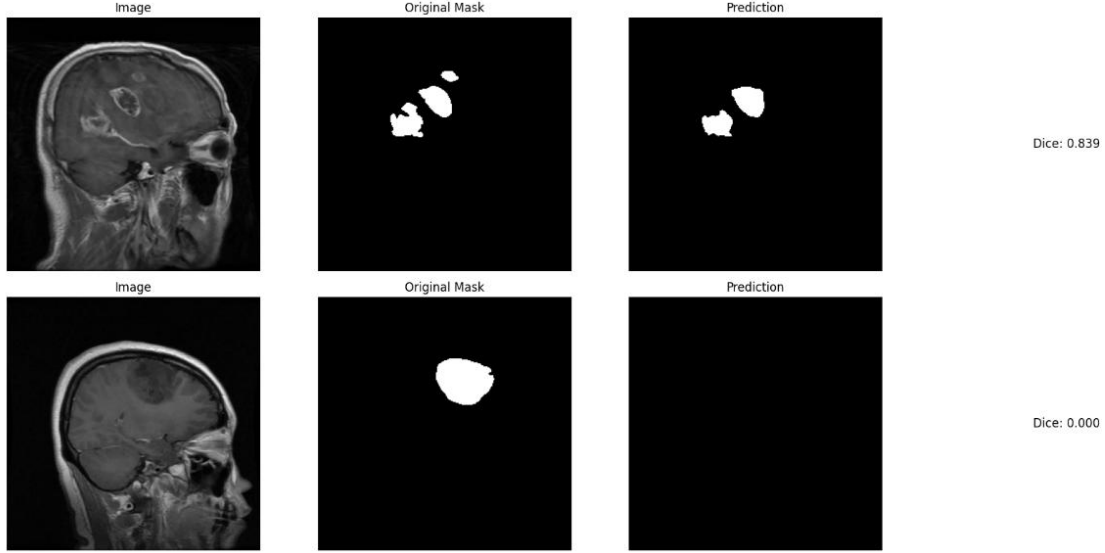


Figure 7: Prediction from Classic U-Net

The classic U-Net often gives tumor boundaries that are not very sharp (Ronneberger et al., 2015). It sometimes fails to detect small tumor areas (Tajbakhsh et al., 2018). The model with MobileNetV2 backbone produces masks that look smoother and more uniform (Sandler et al., 2018).

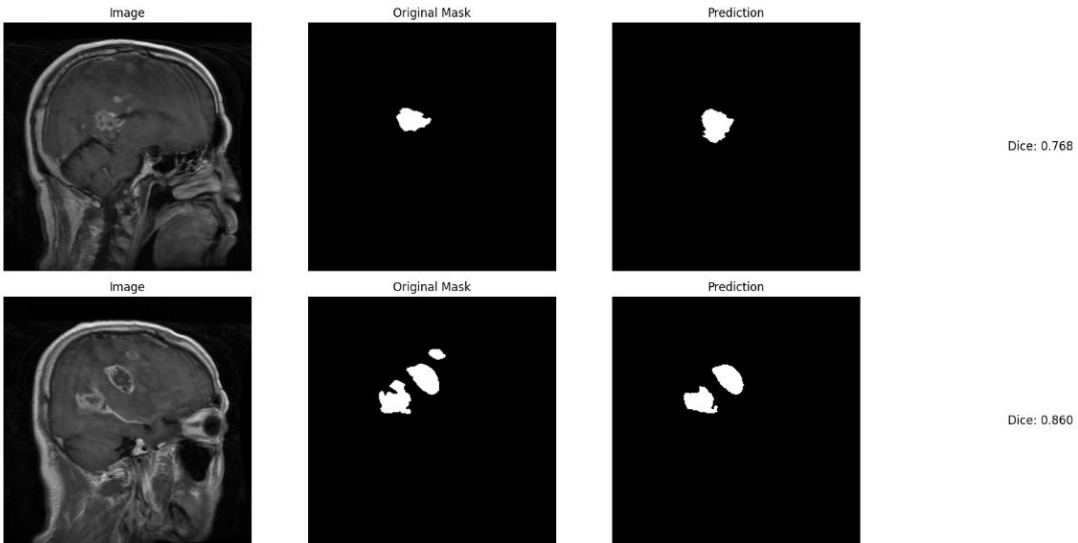


Figure 8: Prediction from MobileNetV2-based U-Net

The model with ResNet50 backbone captures complex tumor shapes and fine boundary details in the most accurate way (He et al., 2016).

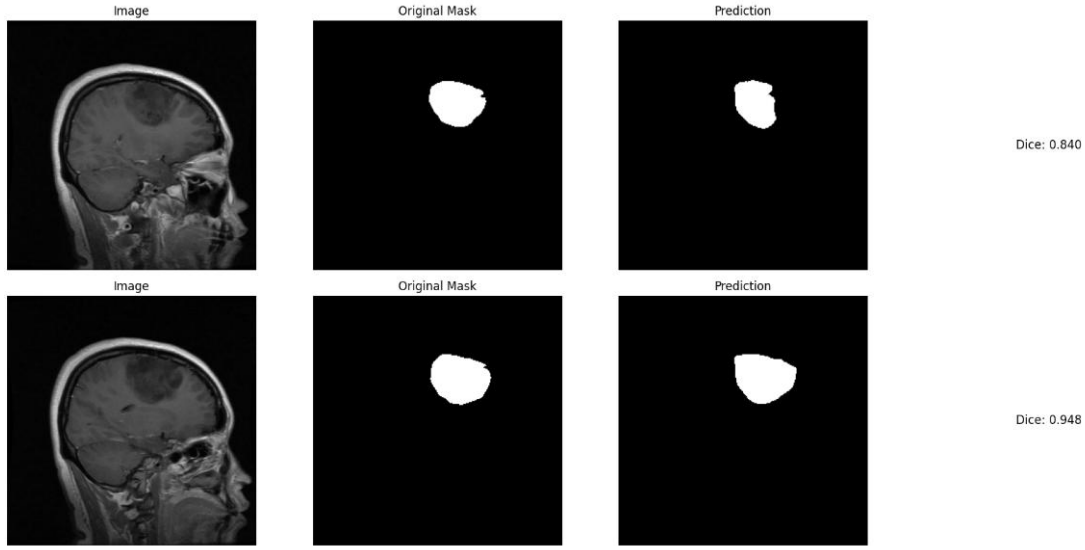


Figure 9: Prediction from ResNet50-based U-Net

These visual results match the quantitative numbers (Tajbakhsh et al., 2018). They show that using pretrained encoders helps get better boundary accuracy and makes the tumor segmentation look more consistent (Tajbakhsh et al., 2018)

7.6. Comparison with other works

Direct comparison with other papers is difficult because many use multi-class segmentation, different datasets, or evaluation methods. On the BRISC dataset, some transformer-based models like SaberNet achieve higher weighted mIoU (around 80.6%) (Fateh et al., 2025). Our best IoU is 78.9%, which is competitive. The difference likely comes from the use of transformer encoders and multi-scale feature fusion in those models, as well as possible variations in how metrics are averaged (weighted across classes vs. binary overlap) (Adoui et al., 2019).

In broader medical segmentation, DeepLabV3+ is a strong baseline, and transformer U-Net variants (such as TransUNet or Swin-UNet) are common modern approaches. Many works focus on overlap metrics and sensitivity to lesions. Adding attention mechanisms can improve recall with little extra cost (Adoui et al., 2019).

Our results show that pretrained encoder backbones provide a simple and effective way to improve performance while keeping trade-offs clear between accuracy and practical deployment constraints (Milletari et al., 2016).

8. CONCLUSION

This project builds a complete pipeline for automatic brain tumor segmentation from MRI images using the BRISC 2025 dataset. We compare three U-Net based models in the same setup: the classic U-Net, U-Net with MobileNetV2 encoder, and U-Net with ResNet50 encoder (Ronneberger et al., 2015; Sandler et al., 2018; He et al., 2016). Replacing the original encoder with a pretrained backbone gives clear benefits. The ResNet50 version achieves the highest Dice (0.882) and IoU (0.789) on the test set. The MobileNetV2 version reaches close performance (Dice 0.869). But uses far fewer parameters and has the fastest inference time (5.72 ms per image). This makes it a practical choice when model size or speed matters for real-world use (Abdelrahman and Viriri, 2023; Milletari et al., 2016).

There are some limitations in this work. We use only one train-validation-test split from the dataset, so we do not test how the models perform on images from different scanners or hospitals. All segmentation is done in 2D slices, so we do not enforce consistency across the full 3D volume of a scan. The latency numbers come from this specific TensorFlow/Keras setup on an NVIDIA A100 GPU; results may change on other hardware or frameworks.

For future work, we can test the models on data from multiple sites to check robustness against scanner and protocol differences (Jiang et al., 2019). Fateh et al. (2025) propose that we can also try transformer-based U-Net models or add attention mechanisms to see if they improve results further and close the gap to the best reported scores on BRISC. When needed, we can extend the approach to 2.5D or full 3D segmentation. To better handle volumetric consistency in clinical applications (Milletari et al., 2016).

Overall, using pretrained encoders with U-Net is a simple and effective way to balance segmentation quality. But there are practical constraints like model size and inference speed (Milletari et al., 2016).

9. REFERENCES

1. Abdelrahman, A. and Viriri, S. (2023) 'EfficientNet family U-Net models for deep learning semantic segmentation of kidney tumors on CT images', *Frontiers in Computer Science*, 5. Available at: <https://doi.org/10.3389/fcomp.2023.1235622>
2. Adoui, M., Mahmoudi, S., Larhman, M.A. and Benjelloun, M. (2019) 'MRI breast tumor segmentation using different encoder and decoder CNN architectures', *Computers*, 8(3), 52. Available at: <https://doi.org/10.3390/computers8030052>
3. Fateh, A., Rezvani, Y., Moayedi, S., Rezvani, S., Fateh, F., Fateh, M. & Abolghasemi, V. (2026) BRISC: Annotated Dataset for Brain Tumor Segmentation and Classification. arXiv preprint arXiv:2506.14318, Available at: <https://arxiv.org/abs/2506.14318>
4. Hussain, T. and Shouno, H. (2024) 'MAGRes-UNet: Improved medical image segmentation through a deep learning paradigm of multi-attention gated residual U-Net', *IEEE Access*, 12, pp.40290–40310. Available at: <https://ieeexplore.ieee.org/document/10460541>
5. Jiang, J., Hu, J., Tyagi, N., Rimner, A., Berry, S., Deasy, J. and Veeraraghavan, H. (2019) 'Integrating cross-modality hallucinated MRI with CT to aid mediastinal lung tumor segmentation', *MICCAI 2019 Lecture Notes in Computer Science*, 11769, pp.221–229. Available at: https://doi.org/10.1007/978-3-030-32226-7_25
6. Kaggle. (2025), BRISC 2025, Available at: <https://www.kaggle.com/datasets/briscdataset/brisc2025>
7. Ronneberger, O., Fischer, P. and Brox, T. (2015) 'U-Net: Convolutional networks for biomedical image segmentation', *Lecture Notes in Computer Science*, 9351, pp.234–241. doi:10.1007/978-3-319-24574-4_28
8. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018) 'MobileNetV2: Inverted residuals and linear bottlenecks', *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp.4510–4520. doi:10.1109/CVPR.2018.00474
9. He, K., Zhang, X., Ren, S. and Sun, J. (2016) 'Deep residual learning for image recognition', *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp.770–778. doi:10.1109/CVPR.2016.90
10. Han, S., Mao, H. and Dally, W.J. (2016) 'Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding', *International Conference on Learning Representations*. doi:10.48550/arXiv.1510.00149
11. Prechelt, L. (2012) 'Early stopping — but when?', *Neural Networks: Tricks of the Trade*, Lecture Notes in Computer Science, 7700, pp.53–67.
12. Milletari, F., Navab, N. and Ahmadi, S.-A. (2016) 'V-Net: Fully convolutional neural networks for volumetric medical image segmentation', *Proceedings of 3DV 2016*, pp.565–571. doi:10.1109/3DV.2016.79
13. Tajbakhsh, N., Jeyaseelan, L., Li, Q., Chiang, J.N., Wu, Z. and Ding, X. (2018). Embracing imperfect datasets: A review of deep learning solutions for medical image segmentation. *Medical Image Analysis*, 63, 101693. <https://doi.org/10.1016/j.media.2020.101693>

10. LIST OF TABLES AND FIGURES

Table 1: Results on the test set along with scalability measures.....	15
Figure 1: Proposed methodology of the project.....	9
Figure 2: Augmentation Data Samples	10
Figure 3: Training Dice Coefficient.....	17
Figure 4: Validation Dice Coefficient.....	17
Figure 5: Training Loss.....	18
Figure 6: Validation Loss.....	18
Figure 7: Prediction from Classic U-Net	19
Figure 8: Prediction from MobileNetV2-based U-Net	19
Figure 9: Prediction from ResNet50-based U-Net.....	20

APPENDIX

- Complete Notebook is available here: [Google Colab](#)
- The source of dataset: [Kaggle](#)